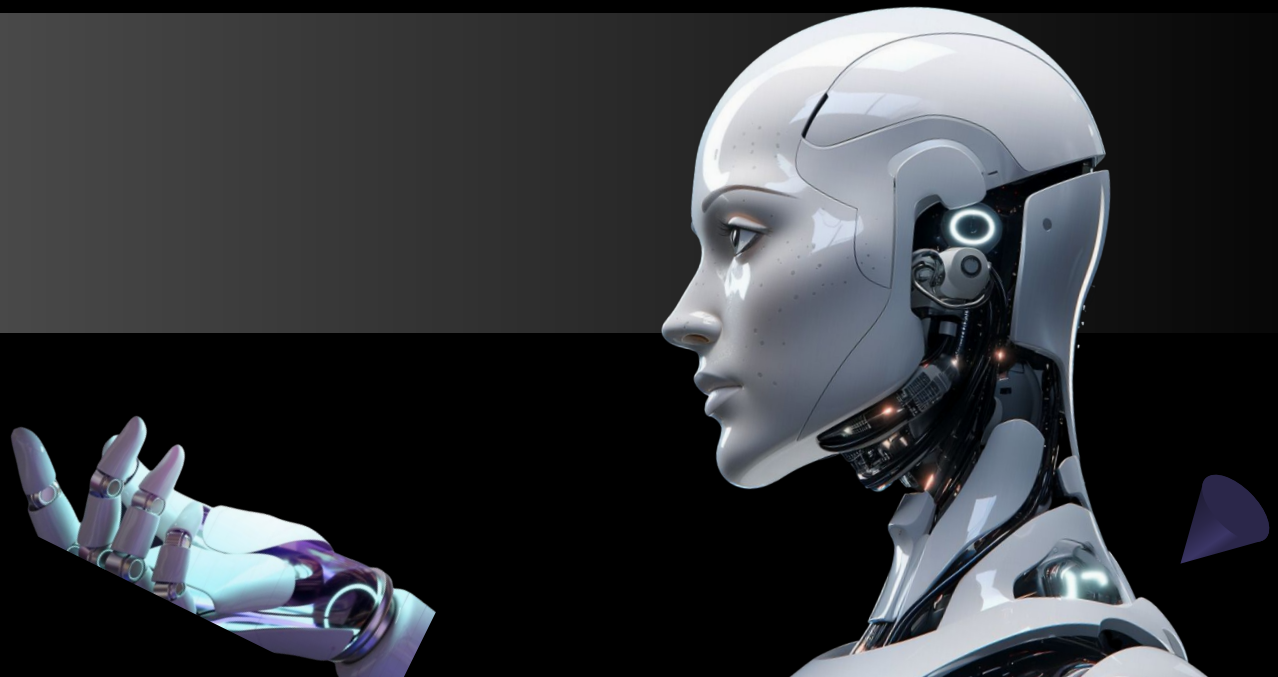
PROJETO EDUCAÇÃO

# DO FUTURO

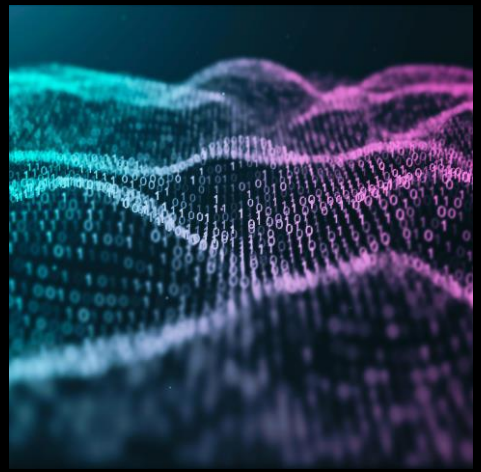
## AULA 01

## Funções



# Sumário

- Programação Procedural
- Funções na Programação
- Sintaxe
- Invocação de uma função
- Função X Procedimento
- Atividades
- Escopo de Variáveis
- Atividades
- Parâmetros Opcionais
- Parâmetros Nomeados
- Atividades
- Desafio



# Programação Procedural

- É uma forma de desenvolvimento em que todos os programas possíveis podem ser reduzidos a apenas três estruturas: **sequência, decisão** e **iteração**.

- Programas feitos com sub-rotinas ou funções são programados e testados uma só vez, embora possam ser usados em várias partes do programa quantas vezes forem necessários.

- Pode-se criar uma biblioteca com as funções e subrotinas, que podem ser usadas em outros programas ou por outros programadores.

- Economia de memória do computador, uma vez que o módulo utilizado é armazenado uma única vez, mesmo que utilizado em diferentes partes do programa.

# Funções na Programação

- Uma **função** é um pedaço de código que faz alguma tarefa específica e pode ser chamado de qualquer parte do programa quantas vezes desejarmos.

Utilizamos funções para obter:

- **Clareza do código**: separando pedaços de código entre funções, podemos entender mais facilmente o que cada parte do código faz.
- **Reutilização:** capacidade de invocar a função em várias partes do seu código.
- **Independência:** uma função é relativamente independente do código que a chamou. Uma função pode modificar variáveis globais ou ponteiros, mas limitando-se aos dados fornecidos pela chamada da função.

# Já utilizamos funções outras vezes...

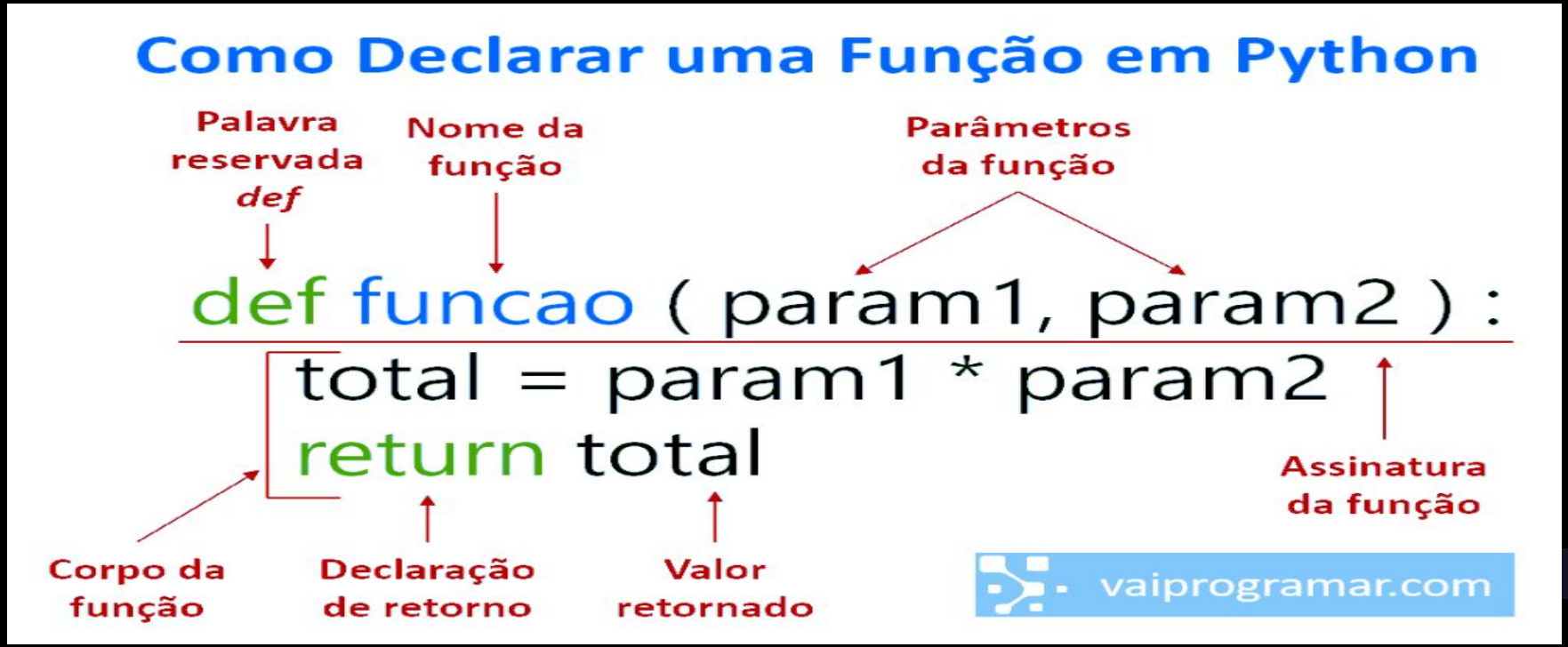
Funções já foram utilizadas por vocês no primeiro dia de aula.

```
print("Olá!")

input("Digite algo :)")

int(input("Função input + função de conversão de tipo inteiro."))
```

# Sintaxe


Como Declarar uma Função em Python
Palavra reservada *def*
Nome da função
Parâmetros da função
def funcao ( param1, param2 ) :
total = param1 * param2
return total
Assinatura da função
Corpo da função
Declaração de retorno
Valor retornado
vaiprogramar.com

# Sintaxe

- **def:** palavra reservada do python para declarar uma função. Usado sempre no início da declaração.
- **função:** nome ou identificador da função. Esse identificador será utilizado para invocar a função em várias partes do seu código.
- **parâmetros:** os parâmetros são definidos após o identificador, por meio dos parênteses. Dentro dos parênteses, serão colocados todos os parâmetros da função, caso exista. Em todo caso, os parênteses são obrigatórios.
- **dois pontos (:):** são utilizados após os parênteses dos parâmetros, indicando abertura de bloco, onde por meio da indentação, serão colocados as instruções da função.

# Sintaxe

- **return:** palavra reservada utilizada no final de uma função para retornar um valor. Para receber esse valor, pode ser utilizado uma variável para armazená-lo.

```python
# Função
def funcao(param1, param2):
  total = param1 * param2

  return total

total = funcao(1, 1)
```

# Invocação de uma Função

- Para invocar uma função, ou seja, executar uma função em uma parte específica do seu código, é necessário usar o identificador dela, acompanhado dos parênteses de abertura e fechamento. Dentro desses parênteses, caso você tenha definido parâmetros na função, será necessário passar os seus respectivos argumentos.

```
funcao(1, 2)
```

# Função X Procedimento

- As funções retornam um valor por meio da palavra reservada (return), por isso, é importante lembrar de atribuir o valor de retorno da função para uma variável.

- Os **procedimentos** (**procedures**) diferem das funções apenas por não retornarem resultado.

- É raro encontrar linguagem que só possua procedimentos (algum dialeto de SQL pode ser, alguma linguagem muito antiga e virtualmente abandonada, ou em algum nicho muito específico).Em algumas poucas linguagens de programação há uma distinção clara entre as duas, outras preferem tratar tudo como funções. Algumas até tratam as funções só como métodos. No fundo, os dois funcionam da mesma forma.

# Função X Procedimento

```python
# Função
def somar(value1, value2):
  return value1 + value2

soma = somar(1, 1)

# Procedimento
def mostrar_bem_vindo():
    print("Bem-vindo :)")

mostrar_bem_vindo()
```

# Atividades

1. Crie um procedimento que quando for chamado apenas mostre a mensagem ‘Olá mundo’.

1. Crie uma função que apenas mostre em tela uma mensagem dinâmica recebida como parâmetro.

1. Faça um programa que tenha uma função chamada área(), que receba à altura e a base de um terreno e calcule a área dele. Fórmula (altura * base)

1. Escreva uma função que recebe dois parâmetros numéricos e imprima o menor dos dois. Se eles forem iguais, imprima que eles são iguais.

1. Escreva uma função que recebe um número qualquer como parâmetro e imprime se esse número é positivo ou negativo.

# Escopo de Variáveis

- O escopo de uma variável indica sua visibilidade – ou seja, a partir de onde, no código, a variável é acessível.

- Temos dois escopos para variáveis em Python:

  - **Escopo Local**: uma variável local (criada dentro de uma função) existe apenas dentro da função onde foi declarada.

  - **Escopo Global**: uma variável global é declarada (criada) fora das funções e pode ser acessada por todas as funções presentes no módulo onde é definida.

# Parâmetros Opcionais

- Às vezes não temos a necessidade de passar um parâmetro a uma função, usando um valor padrão previamente codificado para o parâmetro em questão. Para isso, podemos usar parâmetros opcionais na função – um parâmetro que pode ou não ser passado no momento da chamada à função.

- Se um valor não for passado a um ou mais parâmetros da função, serão utilizados os valores padrão definidos no momento de criação da função em si.

- Para usar o parâmetro opcional, basta usar o sinal de atribuição (=) do lado direito do parâmetro e assim, atribuir um valor padrão.

# Parâmetros Opcionais

```python
# Procedimento
def somar(value_1, value_2, value_3 = 10):
    print(value_1 + value_2 + value_3)

somar(1, 1)
```

# Parâmetros Nomeados

- Em Python é possível invocarmos funções passando os argumentos de forma posicional ou então, através da associação chave-valor.
    - **Argumento posicional:** é o nome utilizado para a passagem de valores seguindo a ordem dos parâmetros conforme foi implementado na função.
    - **Argumento nomeado:** é a passagem de valores fazendo associação com o nome do parâmetro e o valor que está sendo enviado, utilizando o conceito de chave=valor.

# Parâmetros Nomeados

```python
# Passagem de parâmetros posicional
def somar(value_1, value_2, value_3 = 10):
    print(value_1)

somar(1, 3)

# Passagem de parâmetros nomeados
def somar(value_1, value_2, value_3 = 10):
    print(value_1)

somar(value_2=1, value_1=3)
```

# Atividades

1. Faça uma função chamada de ‘média’ que receba 4 notas e faça a média dessas notas.

2. Faça uma função que receba três números e veja qual deles é o maior

1. Faça uma função chamada ‘tabuada’ que recebe um número como parâmetro e mostre toda a tabuada de multiplicação daquele número de 1 até 10. (Faça com for)

1. Faça um função que receba um número que será considerado o raio de um círculo e com isso calcule à área de um círculo.
   Fórmula da área do círculo: A = 3.14 * raio ^ 2

**Obrigado pela atenção! Até a próxima :)**